

# Deepfake Detection and Generation using the DeepFakeFace Dataset

Adriano Machado, Francisco da Ana & Tomás Vicente  
Faculty of Engineering & Faculty of Science, University of Porto

## Abstract

This project focuses on developing deep learning models for both deepfake detection and generation using the DeepFakeFace dataset. For detection, we evaluated transfer learning models (ResNet, EfficientNet, DenseNet, and ViT) alongside custom CNN architectures, achieving over 98% accuracy. For generation, we implemented GANs (MLP-GAN and DCGAN) as well as a Diffusion Model — the latter demonstrating the greatest potential for creating diverse facial features.

## 1 Dataset Analysis

The dataset used in this project is the DeepFakeFace (DFF) dataset, introduced by Köksal et al. (2023). An analysis of image dimensions revealed a significant difference between real and fake images, as shown in Figure 1. All fake images possess uniform dimensions of  $512 \times 512$  pixels (aspect ratio 1.0) while the real images exhibit considerable variation in both width and height leading to a diverse distribution of aspect ratios.

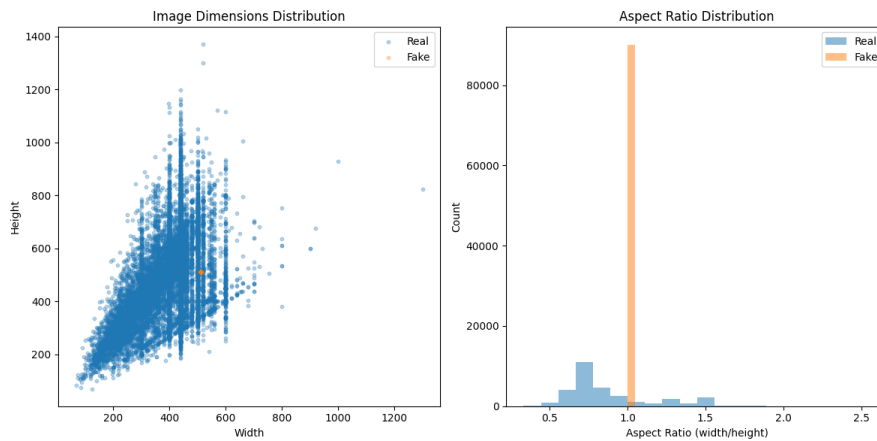


Figure 1: Image dimensions and aspect ratio distribution within the DeepFakeFace dataset

The dataset is structured as follows:

- **30,000 Real Images:** Sourced from the IMDB-WIKI dataset, depicting celebrity faces.
- **90,000 Fake Images:** Generated using three distinct diffusion-based models (30,000 images per model):
  - Stable Diffusion v1.5 (Text-to-Image)
  - Stable Diffusion Inpainting
  - InsightFace Toolbox

The dataset presents a significant class imbalance, with a 3:1 ratio of fake to real images (90,000 fake vs. 30,000 real). This imbalance can bias classifiers towards the majority class (fake), potentially affecting their ability to correctly identify real images. To mitigate this, and considering the computational constraints faced during model training, for our classification task we adopted an **undersampling** strategy by utilizing only 30,000 fake images (10,000 from each type).

Furthermore, the dataset displays substantial visual heterogeneity. Images vary in color properties (full color and black-and-white), tone, saturation, and lighting conditions, a factor that helps explain the variations observed in the outputs from the image generation models.

## 2 Classification Task

The primary objective of this task was to develop a classifier capable of accurately distinguishing between real and fake facial images from the DFF dataset. We explored two main approaches: using pre-trained models via transfer learning and training CNNs from scratch.

### 2.1 Transfer Learning

This approach uses models pre-trained on large-scale datasets, particularly the ImageNet dataset (Deng et al. 2009). The fundamental concept is that these models have already developed robust hierarchical features for general image understanding, which can be fine-tuned for our specific deepfake detection task. This typically results in better performance than training from scratch, especially with limited data, and significantly reduces training time. Choosing an appropriate pre-trained model requires balancing accuracy, computational cost, and model size.

To explore these trade-offs and identify suitable architectures for our task, we evaluated several established models available in PyTorch’s `torchvision.models` library:

- **ResNet** (He et al. 2016): ResNet18, ResNet50
- **EfficientNet** (Tan & Le 2019): EfficientNet\_B0
- **DenseNet** (Huang et al. 2017): DenseNet161
- **Vision Transformer (ViT)** (Dosovitskiy et al. 2020): ViT\_b\_16, ViT\_b\_32

The models were adapted for our binary classification task by modifying their final classification layers to output two classes (real and fake). Specifically, for ResNet and DenseNet, the fully connected layer was replaced with a new linear layer; for EfficientNet, the classifier was modified; and for ViT, the heads were adjusted accordingly.

The dataset was split into training (70%), validation (15%), and test (15%) sets, using cross-entropy loss and optimizers such as Adam or SGD.

#### 2.1.1 ResNet

We conducted a hyperparameter tuning process aiming to find the best combination of model variant, optimizer, and batch size. Due to the lack of computational resources, we limited the tuning to a small number of epochs (5) and not many configurations.

- **Optimizers:** Adam, SGD
- **Batch Sizes:** 64, 128
- **Learning Rate:** 0.001

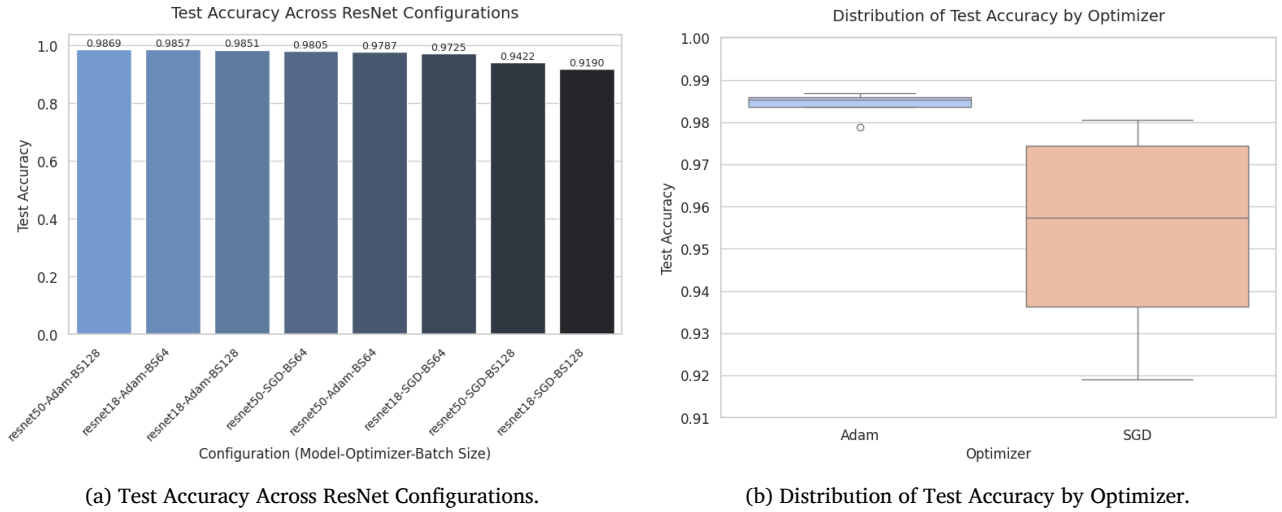


Figure 2: Analysis of ResNet Performance.

Figure 2a shows that the Adam optimizer consistently outperforms SGD across all ResNet configurations, with ResNet50-Adam-BS128 achieving the highest accuracy (98.69%). Moreover, ResNet50 variants generally perform better than ResNet18 under the same settings. Figure 2b presents a boxplot confirming Adam’s superiority, demonstrating both a higher median accuracy (98.5%) and significantly less variance compared to SGD’s wider performance distribution (92%–98%).

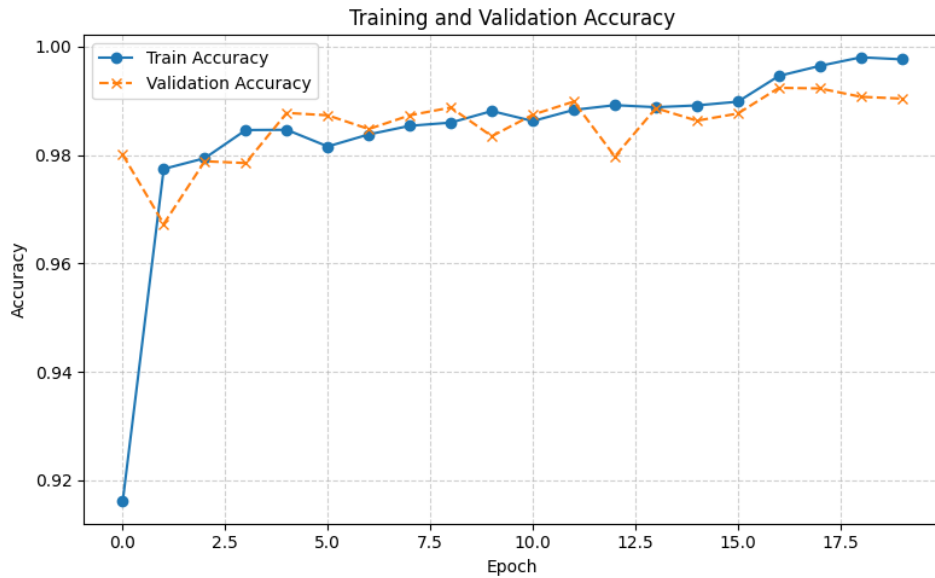


Figure 3: Training and validation accuracy of ResNet50-Adam-BS128.

The selected ResNet50-Adam-BS128 configuration was trained for **20 epochs**, achieving **98.7% test accuracy** and a **near-perfect ROC curve**. Both training and validation accuracy increased significantly in the early epochs. After peaking around epochs 16–17, validation accuracy leveled off, indicating slight overfitting. Overall, the model demonstrated strong performance.

### 2.1.2 EfficientNet\_B0

We evaluated the smallest EfficientNet variant, EfficientNet\_B0, pre-trained on ImageNet. The model was trained for 10 epochs using the Adam optimizer (learning rate = 0.001, batch size = 32) with cross-entropy loss. EfficientNet\_B0 delivered great results, achieving a **test**

**accuracy of 99.55%** — making it our **top-performing model**. Both the ROC AUC curve (see Figure 4b) and the confusion matrix (see Figure 4c) confirm its exceptional capability in distinguishing between real and fake images.

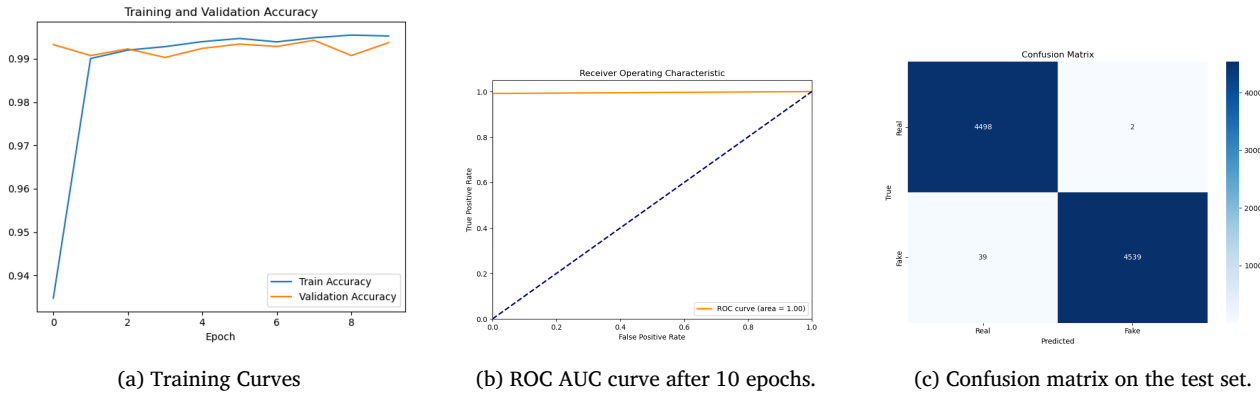


Figure 4: Performance metrics for EfficientNet\_B0.

### 2.1.3 DenseNet

We experimented with DenseNet161 Training was performed using the Adam optimizer (learning rate 0.001), a batch size of 16, and Cross-Entropy loss for 10 epochs.

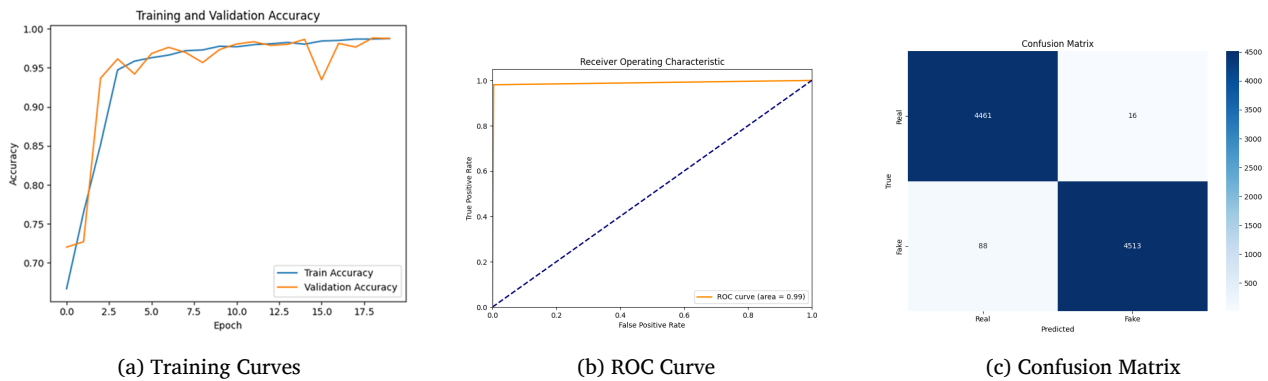


Figure 5: Performance of DenseNet161 (Test Accuracy: 98.85%).

DenseNet161 also achieved excellent results, reaching a test accuracy of 98.85%.

### 2.1.4 Vision Transformer (ViT)

We also evaluated two variants of the Vision Transformer: ViT\_b\_16 and ViT\_b\_32. Using Adam optimizer and batch size 64, ViT\_b\_16 achieved 93.74% test accuracy after 10 epochs, while ViT\_b\_32 reached 87.82%. While good, their performance lagged behind the top CNN-based transfer learning models in this task within the limited training duration.

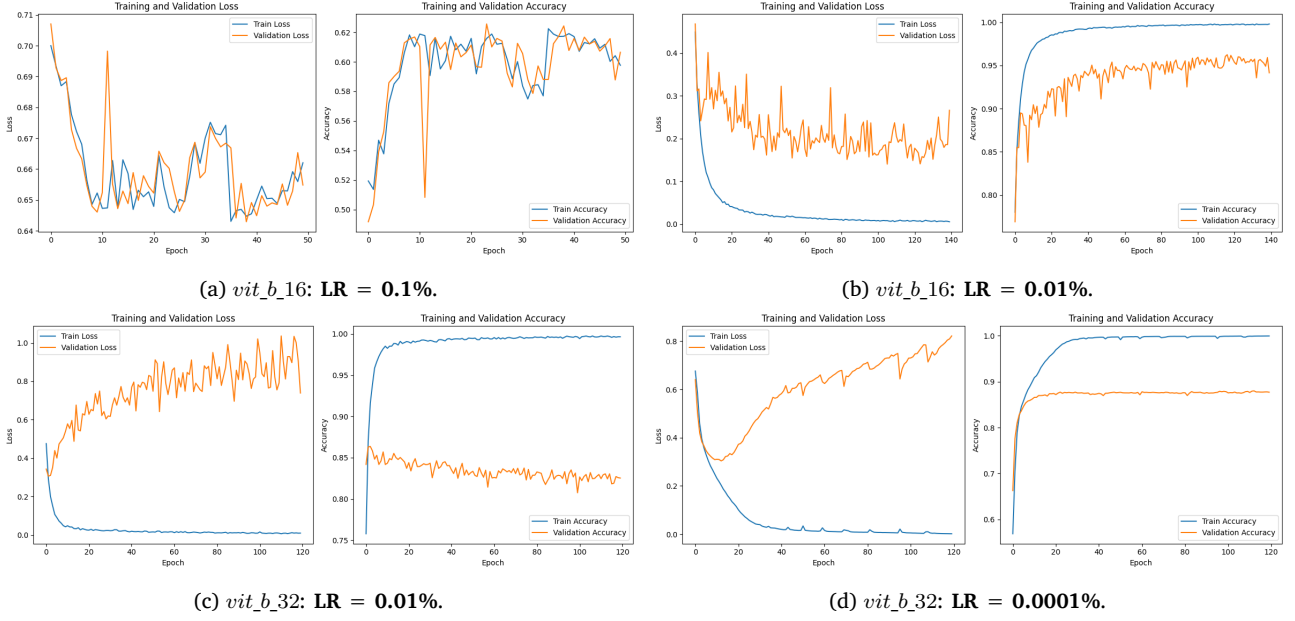


Figure 6: Model Performance related to Learning Rate Variance.

To achieve convergence during training, we observed a necessary reduction in the learning rate compared to typical values. For Visual Transformers utilizing 16x16 pixel patches, we decreased the learning rate to 0.01% (from a more conventional starting point, which was initially explored at 0.1%). Furthermore, when employing larger 32x32 pixel patches, a significantly lower learning rate of 0.0001% was required.

This adjustment is attributed to a combination of factors: the reduced batch size necessitated by local computational limitations, the inherently larger parameter space of Transformer models, and the impact of patch size on feature extraction.

Specifically, larger 32x32 patches can be less effective at capturing fine-grained image details compared to 16x16 patches. This potentially results in a more complex and sensitive loss landscape, where larger learning rates are more likely to induce oscillations and hinder convergence towards optimal minima.

Lastly, the accuracy of the model in itself was also be harmed by the patch size, like we can verify on the graphs.

## 2.2 Custom CNN Architectures

We designed and trained two custom CNN architectures from scratch.

### 2.2.1 Baseline CNN

A simple architecture with 3 convolutional blocks followed by fully connected layers. Each block consists of a convolution layer followed by ReLU activation function and max pooling. From block to block, the number of channels is doubled and the image sizes are reduced to half.

Trained with optimizer Adam, LR=0.001 and batch size 64 for 150 epochs, it achieved a test accuracy of 71.70%.

### 2.2.2 Improved CNN

A deeper architecture incorporating techniques like batch normalization, with more convolutional blocks and deeper MLP for the final classification task.

Trained under similar conditions, but with many more epochs (100) since it was trained from scratch, it significantly improved performance, reaching 98.35% test accuracy, comparable to the transfer learning models.

The model was also tested using a single output perceptron with a sigmoid activation function for binary classification. The goal was to assess whether modifying the architecture of the output layer would lead to different results. However, the performance remained consistent with the previous configuration, achieving a test accuracy of 99.03%.

### 2.2.3 Training results and comparison

Figure 7 illustrates the evolution of training and validation loss and accuracy of both custom CNNs. By analyzing these plots we can see the impact of the improved architecture in the classification task.

These plots reveal an interesting insight: there is little evidence of overfitting during the training of the baseline model. Although the validation accuracy remains stable across epochs, the validation loss increases significantly while the training loss continues to decrease. This suggests that the model is still making correct predictions — the most probable class is often the correct one — but with decreasing confidence. In other words, the model's predictions for the correct class are becoming less decisive, leading to higher loss values. This indicates a growing overlap or confusion between the classes, even though the overall classification performance (in terms of accuracy) does not degrade.

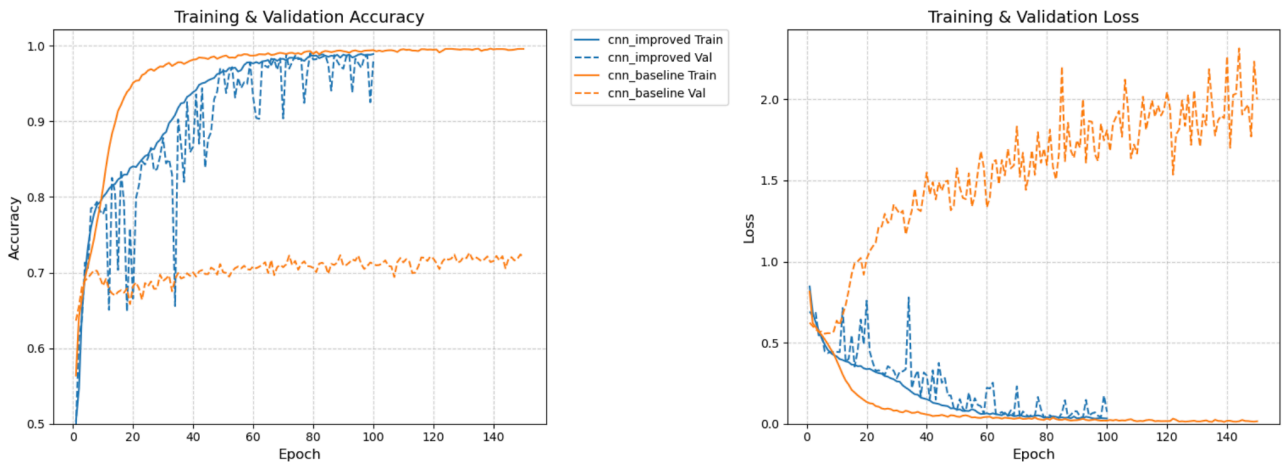


Figure 7: Training dynamics of custom CNNs

## 2.3 Comparative Analysis of Classifiers

| Model           | Test Accuracy (%) |
|-----------------|-------------------|
| ResNet50        | 98.70             |
| EfficientNet_B0 | 99.55             |
| DenseNet161     | 98.85             |
| ViT_b_16        | 93.74             |
| ViT_b_32        | 87.82             |
| Baseline CNN    | 71.70             |
| Improved CNN    | 98.35             |

Table 1: Comparison of classifier performance based on test accuracy.

Transfer learning models demonstrated rapid convergence within few epochs and excelled at deepfake detection, led by EfficientNet\_B0 (99.55% accuracy), DenseNet161 (98.85%), and ResNet50 (98.70%). Notably, our custom Improved CNN also performed remarkably well (99.03%), proving highly competitive, unlike the Baseline CNN (71.70%). While the Visual Transformers lagged slightly (16 pixel-sized patch: 93.74%), specially the 32 pixel-sized patch variation (87.82%).

## 3 Generative Task

The second objective of this project was to explore the generation of realistic facial images using models trained on the DeepFakeFace dataset. We investigated two main classes of generative models: Generative Adversarial Networks (GANs) and Diffusion Models.

### 3.1 Generative Adversarial Networks (GANs)

GANs (Goodfellow et al. 2014) consist of a dual-network architecture where two models — the Generator (G) and the Discriminator (D) — engage in a competitive minimax game. The Generator transforms random noise into synthetic data samples that aim to emulate the true data distribution, while the Discriminator evaluates inputs to distinguish between authentic data and G’s generated samples. In this adversarial setting, the Generator’s goal is to produce increasingly realistic outputs that can deceive the Discriminator, which in turn refines its ability to correctly classify real versus fake data. This dynamic process pushes both networks to continuously improve their performance.

#### 3.1.1 MLP-GAN

Our initial generative model was a simple GAN using Multi-Layer Perceptrons (MLPs) for both the generator and discriminator.

- **Generator:** Takes a 100-dimensional latent noise vector and maps it through several fully connected layers with LeakyReLU activations to produce a flattened 12,288-dimensional vector (representing a 64x64x3 image), followed by a Tanh activation to scale outputs to  $[-1, 1]$ .
- **Discriminator:** Takes a flattened image vector, passes it through MLP layers (LeakyReLU, Dropout), and outputs a single probability (via Sigmoid) indicating the likelihood of the input being real.

The MLP-GAN was trained for 1500 epochs using Adam optimizer (LR = 0.0002), batch size 512, and Binary Cross-Entropy (BCE) loss.

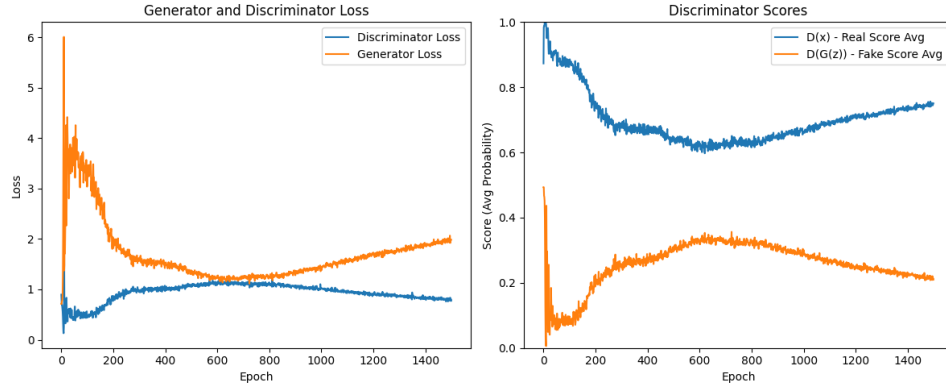
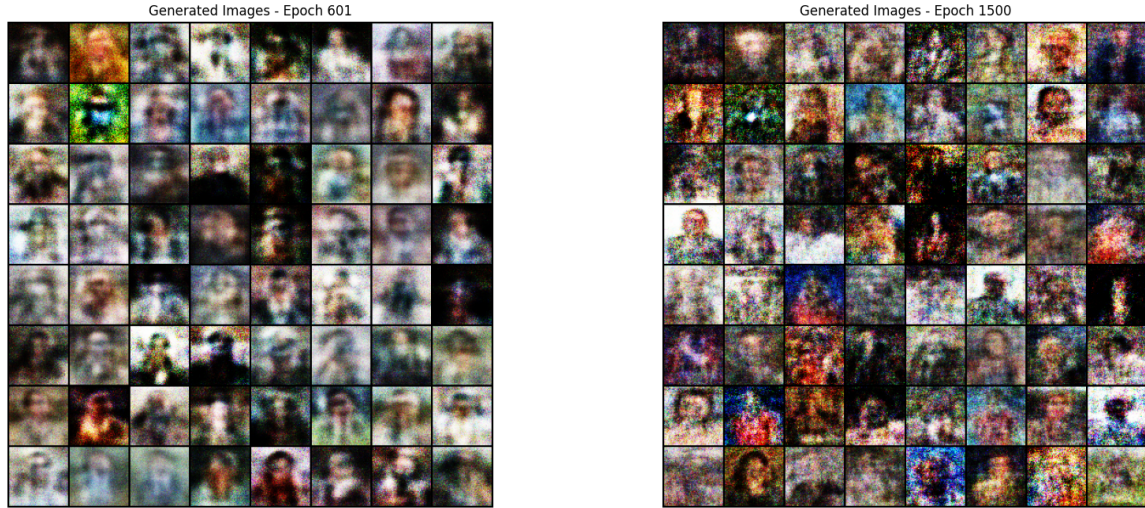


Figure 8: Training Loss Curves of MLP-GAN.

The training curves (Figure 8) show initial volatility followed by a period of relative stability (epochs 600-800), after which the generator loss begins to increase while the discriminator loss continues to decrease. This divergence suggests the discriminator is gaining an advantage over the generator, creating an imbalance in the adversarial training dynamic.



(a) Generated Samples at Epoch 601

(b) Generated Samples at Epoch 1500

Figure 9: MLP-GAN Sample Evolution.

The comparison between samples at epochs 601 (Figure 9a) and 1500 (Figure 9b) provides insight into the model’s performance. Images at epoch 601 show blurry but recognizable facial structures with muted colors. By epoch 1500, while the images display increased color saturation and textural complexity, they paradoxically exhibit a reduction in coherent facial features. This visual degradation aligns with the diverging loss curves, where the increasing generator loss manifests as heightened noise and color artifacts rather than improved facial fidelity.

The MLP-GAN experiment demonstrates both the fundamental capacity of GANs to synthesize images from noise and the limitations of using simple MLP architectures for complex image generation tasks. These findings motivated our exploration of more sophisticated architectures, such as DCGAN.

### 3.1.2 Deep Convolutional GAN (DCGAN)

To better handle spatial image features, we implemented a DCGAN (Radford et al. 2015), replacing MLPs with convolutional layers.

- **Generator:** Uses ConvTranspose2d layers with BatchNorm2d and ReLU (or LeakyReLU in a variant) to upsample a 100D latent vector into a 64x64x3 image (output via Tanh).
- **Discriminator:** Uses Conv2d layers with BatchNorm2d (optional) and LeakyReLU to downsample an input image to a single probability (output via Sigmoid).

We initialized weights with mean 0 and standard deviation 0.02 as suggested in the DCGAN paper and used Adam optimizer (LR = 0.0002), BCE loss, and batch size 2048. We trained a variant using LeakyReLU in the generator for 1500 epochs. In terms of image transformations we resized the images to 64x64, center-cropped to ensure consistent dimensions and normalized to the range [-1, 1].

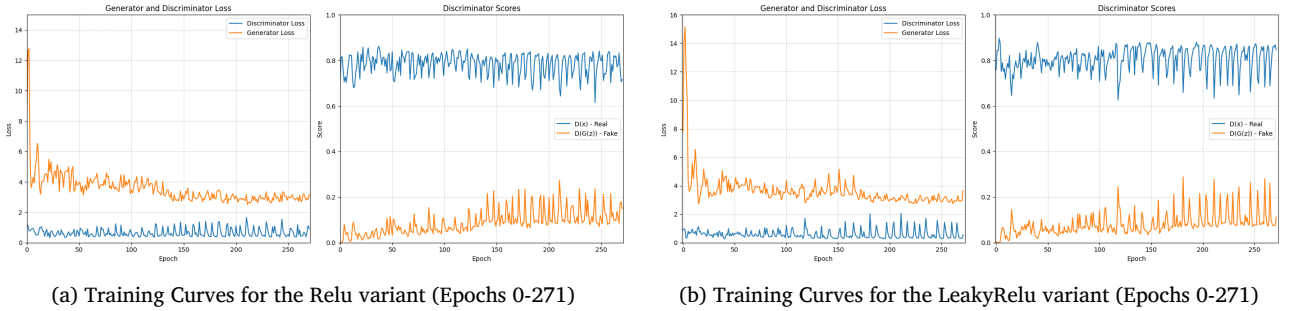
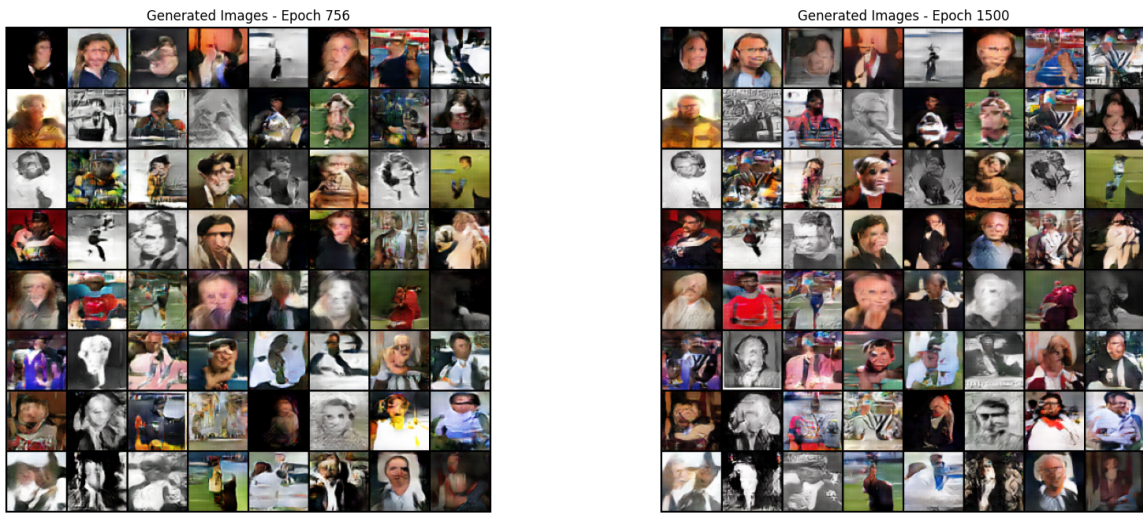


Figure 10: DCGAN Training Loss Curves.

We continued training the LeakyReLU variant for a total of 1500 epochs. The generated samples (Figure 11) show clearer spatial structures and more defined face-like shapes than the MLP-GAN. Yet, the quality remains limited by the 64x64 resolution, and images suffer from artifacts and distortions.



(a) Generated Samples at Epoch 756

(b) Generated Samples at Epoch 1500

Figure 11: DC-GAN Sample Evolution.



(a) Epoch 756

(b) Epoch 1500

Figure 12: Upclose Face at Epochs 756 and 1500.

Comparing samples from epoch 756 (Figure 11a) to 1500 (Figure 11b) does not reveal a dramatic improvement. In fact, as seen in the close-ups (Figure 12), some faces appear to have deteriorated slightly, exhibiting increased distortions and a loss of feature clarity. This observation, coupled with the rising generator loss shown in Figure 10b, suggests the model might have reached its performance limit for this architecture and resolution.

### 3.2 Diffusion Models

As an alternative to GANs, we explored Denoising Diffusion Probabilistic Models (DDPMs) (Ho et al. 2020). These models learn to generate data by reversing a gradual noising process.

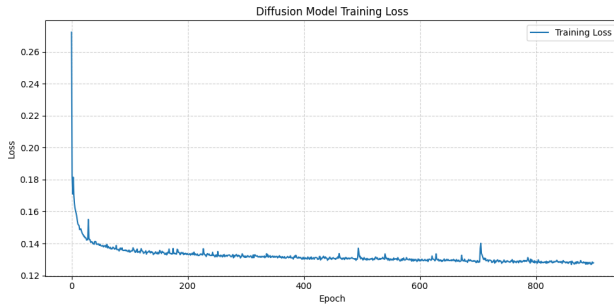
#### Concept:

1. **Forward Process:** Gradually add Gaussian noise to an image  $x_0$  over  $T$  timesteps, producing noisy versions  $x_1, \dots, x_T$ .  $x_T$  is essentially pure noise. This process is fixed and defined by a variance schedule  $\beta_t$ .
2. **Reverse Process:** Train a model (in our case a U-Net) to predict the noise added at each step  $t$ . Starting from pure noise  $x_T$ , the model iteratively denoises it using the predicted noise ( $\epsilon_\theta(x_t, t)$ ) to step back towards  $x_0$ , generating a sample.

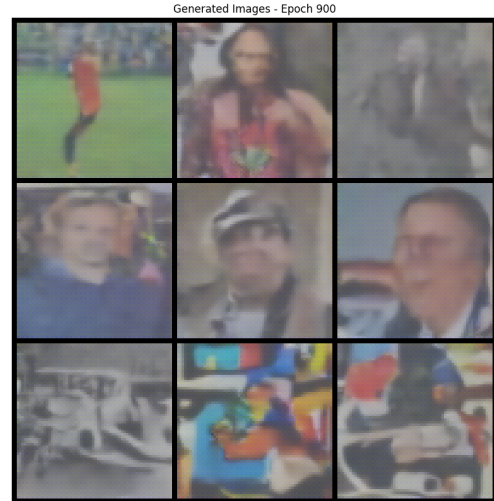
#### 3.2.1 Initial Attempt (Custom U-Net)

To gain a practical understanding of diffusion models, we implemented a simplified U-Net architecture based on a helpful tutorial by Florian Wilhelm ( DeepFindr).

- **Architecture:** A custom U-Net with 5 down/up blocks (Channels: 64 to 1024 and back), incorporating sinusoidal time embeddings (32-dim).
- **Diffusion Parameters:**  $T = 300$  timesteps, linear beta schedule ( $\beta_1 = 0.0001$  to  $\beta_T = 0.02$ ).
- **Training:** Trained for 900 epochs on 64x64 real images using Adam optimizer (LR = 0.001), batch size 128, and L1 loss.



(a) Training Loss Curve (L1 Loss)



(b) Generated Samples at Epoch 900

Figure 13: Initial Diffusion Model (Custom U-Net, 64x64) Training Loss and Samples.

The training loss (Figure 13a) indicated successful learning. The generated samples at epoch 900 (Figure 13b) showed diverse, recognizable face-like structures, capturing variations from the dataset. However, the images suffered from low resolution (64x64) and tended to have a desaturated, somewhat grayish appearance, lacking vibrant colors.

### 3.2.2 Second Attempt (Hugging Face Diffusers Library)

To improve visual quality and leverage standardized, optimized components, we implemented a second diffusion model using the Hugging Face ‘diffusers’ library. This allowed for easier integration of standard architectures and training techniques.

- **Architecture:** Utilized the standard ‘UNet2DModel’ from ‘diffusers’ configured for 128x128 image generation (block channels: (128, 128, 256, 256, 512, 512), incorporating attention layers).
- **Scheduler:** Adopted the ‘DDPMScheduler’ with 1000 training timesteps and a ‘squared-cos\_cap\_v2’ beta schedule.
- **Training:**
  - **Optimizer:** AdamW (LR = 1e-4).
  - **LR Schedule:** Cosine schedule with warmup (500 steps).
  - **Image Size:** 128x128 pixels.
  - **Batch Size:** 16 (per device).
  - **Loss:** Mean Squared Error (MSE) between predicted noise and actual noise (standard for ‘diffusers’ training loop).
  - **Duration:** Trained for 30 epochs due to significant computational cost.



Figure 14: Generated Samples (128x128) after 30 Epochs using the ‘diffusers’ library implementation.

The results from this second approach, even after only 30 epochs (Figure 14), showed a marked improvement in visual quality compared to the initial attempt. The generated 128x128 images exhibit more vibrant colors and potentially better-defined structures, although still abstract and far from photorealistic. However, this improvement came at a significant computational cost: training for 30 epochs required approximately 10 hours on the available hardware. This constraint prevented us from training the model for the planned number of epochs (e.g., 50-100 or more).

Overall, our experiments with diffusion models highlighted their potential for generating diverse facial images that capture dataset characteristics. While the initial custom implementation provided a baseline, using the ‘diffusers’ library led to better visual quality.

## 4 Discussion and Conclusions

This project successfully explored both deepfake detection and generation using the DeepFakeFace dataset.

**Deepfake Detection:** Our classification experiments showed that diffusion-based fakes in this dataset were highly detectable. Transfer learning models performed exceptionally well - EfficientNet\_B0 reached 99.55% accuracy, with DenseNet161 (98.85%) and ResNet50 (98.70%) close behind, all with relatively brief training. Notably, our custom “Improved CNN” achieved 98.35% accuracy when trained from scratch, demonstrating that it’s possible to deliver strong results even without pre-training.

### Image Generation:

Our generative experiments revealed strengths and limitations across different approaches, though we were constrained by computational resources, particularly affecting resolution and training time. The baseline MLP-GAN struggled with spatial relationships. The DCGAN (64x64) showed improvement but wasn’t able to completely capture the details. Our first Diffusion Model (DDPM) implementation at 64x64 resolution showed promising diversity but generated desaturated images. When we switched to the diffusers library framework at 128x128 resolution, we achieved better visual quality and color reproduction after just 30 epochs. However, the substantially higher computational demands (10 hours for 30 epochs) prevented the longer training needed for truly high-fidelity results. Among all approaches

tested, the diffusers-based DDPM showed the most potential for quality and diversity, despite its resource requirements.

**Limitations and Future Work:** The primary limitation was computational resources, restricting the extent of hyperparameter tuning, the training duration (especially for the ‘diffusers’ model) and the resolution for generative models. The undersampling in classification, while effective, discarded a large portion of the fake data. Future work could involve:

- Train generative models at higher resolutions (128x128 or 256x256) for more epochs.
- Conducting more extensive hyperparameter optimization for all models.
- Exploring techniques to handle class imbalance without discarding data (class weighting, oversampling).
- Testing the generalizability of the best detection models on other deepfake datasets or real-world examples.

## References

- (DeepFindr), Florian Wilhelm. 2022. Diffusion models from scratch in pytorch. [https://www.youtube.com/watch?v=a4Yfz2FxxiY&t=176s&ab\\_channel=DeepFindr](https://www.youtube.com/watch?v=a4Yfz2FxxiY&t=176s&ab_channel=DeepFindr). YouTube video, published July 17, 2022.
- Deng, Jia, Wei Dong, Richard Socher, Li-Jia Li, Kai Li & Li Fei-Fei. 2009. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, 248–255. Ieee.
- Dosovitskiy, Alexey, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit & Neil Houlsby. 2020. An image is worth 16x16 words: Transformers for image recognition at scale.
- Goodfellow, Ian, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville & Yoshua Bengio. 2014. Generative adversarial nets. In *Advances in neural information processing systems (nips)*, .
- He, Kaiming, Xiangyu Zhang, Shaoqing Ren & Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 770–778.
- Ho, Jonathan, Ajay Jain & Pieter Abbeel. 2020. Denoising diffusion probabilistic models.
- Huang, Gao, Zhuang Liu, Laurens Van Der Maaten & Kilian Q Weinberger. 2017. Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 4700–4708.
- Köksal, Mustafa Kaan, Emre Can Kaya, Sefa Koçak, Paulo Korshunov, Romain Negrel, Baptiste Praz, Hamza Abidar & Alexandros Kalousis. 2023. Robustness and generalizability of deepfake detection: A study with diffusion models.
- Radford, Alec, Luke Metz & Soumith Chintala. 2015. Unsupervised representation learning with deep convolutional generative adversarial networks.
- Tan, Mingxing & Quoc V Le. 2019. Efficientnet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning*, 6105–6114. PMLR.